Relatório de Análise de Segurança de Código

1. Informações Preliminares **2**
2. Dados do Analista e da Empresa de Consultoria **3**
3. Metodologia de Análise **4**
4. Sumário Executivo **5**
5. Findings Detalhados e Recomendações **6**
Finding FND-001: Man in the middle (Severidade: Crítica) 6
Finding FND-002: Vulnerabilidades em dependências (Severidade: Alta) 7
Finding FND-003: Configuração de CORS excessivamente permissiva (Severidade: Baixa) 8
Finding FND-004: Risco de Acesso ao AuthFile no EDR AV Scanner (Severidade: Média) 9
Finding FND-005: Falta de testes automatizados (Severidade: Média) 10
Finding FND-006: Falta de configuração para ferramentas de suporte a desenvolvedor (Severidade: Informativa) 11
Finding FND-007: Arquivos fonte extensos (Severidade: Média) 12
Finding FND-008: Falta de uma ferramenta configurável de logging (Severidade: Informativa) 13
Finding FND-009: Validação de Host no sshCollector.sh (Severidade: Informativa) ... 14
6. Conclusão e Próximos Passos **15**

1. Informações Preliminares

Este documento apresenta os resultados da análise de segurança de código-fonte da aplicação SamurEye, desenvolvida pela GruppenIT. O objetivo principal desta análise é identificar vulnerabilidades, falhas de segurança e riscos potenciais no código da aplicação que possam ser explorados maliciosamente, comprometendo a confidencialidade, integridade e disponibilidade dos dados e funcionalidades.

Item	Detalhes
Empresa Contratante / Cliente	GruppenIT
Aplicação Analisada	SamurEye
Desenvolvedor(es)	GruppenIT
Versão da Aplicação / Componente	a232faa064fa88fb8e97e3c7eda0bbf467211190
Data de Início da Análise	23 de fev. de 2026
Data de Conclusão da Análise	8 de mar. de 2026
Elaborado por	Rodrigo Urubatan Ferreira Jardim

2. Dados do Analista e da Empresa de Consultoria

O relatório foi elaborado por um especialista sênior com mais de 27 anos de vasta experiência em segurança de redes, desenvolvimento de sistemas com foco em segurança e disponibilidade.

Item	Detalhes
Nome do Analista Principal	Rodrigo Urubatan Ferreira Jardim
Credenciais / Cargo	Especialista Sênior em Segurança de Código e Sistemas
Empresa de Consultoria	*RODRIGO URUBATAN FERREIRA JARDIM CONSULTORIA EM TECNOLOGIA DA INFORMACAO LTDA*
Contato (Email)	urubatan@gmail.com
Contato (Telefone)	(51)992561542

3. Metodologia de Análise

A segurança do código-fonte foi avaliada por meio de uma abordagem híbrida. Utilizou-se a combinação de ferramentas automatizadas de Análise Estática de Aplicações (SAST) e uma revisão manual detalhada, seguindo padrões e melhores práticas de segurança de código (como o OWASP ASVS). O objetivo principal foi identificar vetores de ataque comuns e vulnerabilidades específicas à linguagem e ao ambiente da aplicação SamurEye.

O processo de análise envolveu:

- **Análise Estática e Dinâmica de Código:** Uso de ferramentas específicas para SAST e Análise Dinâmica de Aplicações (DAST).
- **Revisão Manual de Código:** Verificação minuciosa do código-fonte.
- **Teste de Penetração (PenTest):** Execução de testes com o aplicativo rodando em um ambiente de máquina virtual.

Adicionalmente, foram realizadas simulações de ataques avançados:

- **Simulação de DNS Spoofing:** Ataque de falsificação de DNS.
- **Simulação de Ataque Man-in-the-Middle (MITM):** O servidor de dashboard legítimo da GruppenIT foi substituído por um servidor falso. Este servidor falso simulou o ataque, enviando comandos ao servidor principal por meio de sintaxe inválida em variáveis de ambiente de atualização.

4. Sumário Executivo

A análise de segurança de código da aplicação SamurEye revelou uma excelente qualidade e organização, com a ausência de problemas de segurança graves de fácil exploração. A única vulnerabilidade de alto risco identificada requer que a rede do cliente já esteja comprometida para ser explorada.

Este relatório apresenta pontos de atenção e melhorias sugeridas. Embora nenhuma das descobertas impeça o uso da aplicação em ambientes de produção, a implementação das recomendações é crucial para elevar o nível de segurança e facilitar a manutenção. O documento também inclui sugestões de aprimoramento para o ambiente de desenvolvimento.

A análise da aplicação SamurEye resultou na identificação de 9 vulnerabilidades, classificadas nas seguintes categorias de severidade:

Severidade	Número de Findings	Porcentagem
Crítica	1	11%
Alta	1	11%
Média	3	33%
Baixa	1	11%
Informativa	3	33%

As vulnerabilidades estão detalhadas na seção 5, juntamente com as recomendações de remediação.

5. Findings Detalhados e Recomendações

Os findings de segurança são apresentados abaixo, incluindo o impacto e as recomendações de mitigação.

Finding FND-001: Man in the middle (Severidade: Crítica)

Descrição da Vulnerabilidade

O problema de segurança de maior severidade reside na conjunção da ausência de validação de certificados ao estabelecer comunicação com o servidor/dashboard central da GruppenIT e a falta de limitação dos parâmetros recebidos para a configuração de variáveis de ambiente, que são utilizadas diretamente pelo comando de atualização. Esta combinação compromete a aplicação, tornando-a suscetível a um ataque Man-in-the-Middle (MITM) que possa simular o *heartbeat* central e retornar um comando de atualização contendo um *payload* arbitrário.

Recomendação de Mitigação

- **Validação de Certificado:** Implementar a validação do certificado da GruppenIT no `SubscriptionService` para assegurar comunicação direta e autêntica.
- **Validação de Parâmetros:** Validar rigorosamente os formatos dos parâmetros recebidos no `SystemUpdateService`. Caso ocorra um ataque MITM, isso impedirá a execução de comandos parciais, já que os valores recebidos são usados diretamente para compor as *strings* executadas no `update.sh`.
- **Opção Adicional (Assinatura):** Utilizar algum tipo de assinatura criptográfica nos comandos, possivelmente empregando a chave de API do cliente mais um *salt* para assinar os parâmetros retornados, como alternativa para resolver o problema de segurança.

Finding FND-002: Vulnerabilidades em dependências (Severidade: Alta)

Descrição da Vulnerabilidade

Foram detectadas vulnerabilidades em algumas dependências da aplicação.

- **Severidade:** 21 de severidade alta e pelo menos 1 crítica.

Recomendação de Mitigação

Priorizar a resolução imediata, pois, em geral, estas vulnerabilidades são corrigíveis rapidamente através de um `npm audit fix`.

Finding FND-003: Configuração de CORS excessivamente permissiva (Severidade: Baixa)

Descrição da Vulnerabilidade

A implementação de CORS utiliza um callback (`server/index.ts\#16-35`) que permite a comunicação com todos os domínios.

- **Contexto:** Não é crítico em ambientes onde a aplicação roda apenas em rede interna.
- **Risco:** Se um cliente instalar o SamurEye acessível via web, a aplicação pode ficar aberta a potenciais ataques.

Recomendação de Mitigação

Manter como ponto de atenção e restringir os domínios permitidos caso a aplicação seja disponibilizada publicamente.

Finding FND-004: Risco de Acesso ao AuthFile no EDR AV Scanner (Severidade: Média)

Descrição da Vulnerabilidade

No módulo EDR AV Scanner, existe o risco de que, caso a máquina hospede outros softwares além do SamurEye, um programa malicioso possa obter acesso ao `AuthFile` gerado e capturar as credenciais de usuário e senha com privilégios administrativos.

Recomendação de Mitigação

Recomendação Importante: A máquina utilizada para executar o sistema **SamurEye** deve ser dedicada exclusivamente a essa finalidade.

Finding FND-005: Falta de testes automatizados (Severidade: Média)

Descrição da Vulnerabilidade

A carência de testes adequados pode acarretar na introdução de vulnerabilidades futuras, notadamente em cenários de modificações de código assistidas por Inteligência Artificial, comprometendo a integridade e a garantia da não regressão das funcionalidades preexistentes.

Recomendação de Mitigação

Implementar uma cobertura abrangente de testes automatizados é crucial para garantir a **qualidade e a estabilidade contínua** do sistema. O objetivo principal é desenvolver um *framework* robusto de testes que valide **todas as regras de negócio** atualmente em produção, cobrindo cenários de sucesso, de erro e de exceção. Essa iniciativa visa **prevenir a introdução de regressões e defeitos** em funcionalidades já entregues e consideradas estáveis.

A estratégia de testes deve incluir:

1. **Testes Unitários:** Focados em validar a lógica individual de classes e métodos, isolando componentes críticos das regras de negócio.
2. **Testes de Integração:** Para verificar a comunicação e o fluxo de dados entre diferentes módulos, serviços e, especialmente, a interação com o banco de dados e sistemas externos (APIs).
3. **Testes de Regressão (End-to-End):** Simular o fluxo completo do usuário através da interface ou dos principais *endpoints* da API, assegurando que as funcionalidades chave permaneçam operacionais e aderentes às especificações.

Este investimento em automação de testes não só melhora a confiança no processo de *deployment* e nas entregas contínuas (*CI/CD*), mas também **reduz significativamente o custo e o tempo** gasto em testes manuais de regressão a cada ciclo de desenvolvimento.

Finding FND-006: Falta de configuração para ferramentas de suporte a desenvolvedor (Severidade: Informativa)

Descrição da Vulnerabilidade

Não foram identificadas configurações padrão para linters e outras ferramentas de verificação e formatação de código no repositório.

Recomendação de Mitigação

A implementação de configurações padrão para ferramentas de análise estática de código (linters), como o ESLint, é uma medida crucial de engenharia de software. Esta ação é fundamental para **garantir a consistência de padrões de codificação e formatação** em todo o projeto, independentemente de qual desenvolvedor esteja contribuindo.

Em um ambiente de desenvolvimento onde o sistema é mantido por uma **equipe colaborativa com múltiplos desenvolvedores**, a ausência de um padrão unificado pode levar rapidamente à fragmentação do estilo, dificultando a leitura, a manutenção e a revisão do código (code review). A adoção de um arquivo de configuração centralizado (ex: `.eslintrc.json`) torna-se uma **necessidade operacional** que aumenta a **qualidade geral, a previsibilidade e a sustentabilidade** do código-base a longo prazo.

Finding FND-007: Arquivos fonte extensos (Severidade: Média)

Descrição da Vulnerabilidade

Embora não se configure como uma vulnerabilidade de segurança diretamente, foi constatada a existência de pelo menos 6 arquivos com mais de 1000 linhas de código. Arquivos com um elevado número de linhas, comumente denominados *God Objects* ou *God Classes*, tendem a comprometer a manutenibilidade, a legibilidade e a capacidade de testes do sistema, aumentando a complexidade e o risco de introdução de defeitos durante futuras modificações.

Recomendação de Mitigação

A divisão do código por responsabilidade é o caminho ideal a ser seguido, especialmente em arquivos que ultrapassam 1000 ou 3000 linhas de código. Essa prática não é apenas uma preferência estilística, mas uma estratégia fundamental para garantir a saúde e longevidade do projeto.

Vantagens de dividir arquivos extensos (acima de 1000 ou 3000 linhas):

1. **Melhoria Significativa na Manutenibilidade:** Arquivos menores, focados em uma única tarefa ou módulo, facilitam a localização de funcionalidades, simplificam a correção de bugs e reduzem o risco de introduzir novos problemas.
2. **Aumento da Inteligibilidade e Leitura:** Aumenta a velocidade de desenvolvimento e facilita o *onboarding* de novos membros da equipe.
3. **Facilitação de Testes Unitários:** Código bem segregado é mais fácil de isolar e testar.
4. **Otimização do Desenvolvimento Colaborativo (Controle de Versão):** Reduz a frequência e a complexidade de conflitos de *merge* no Git.
5. **Adesão a Princípios de Design de Software:** Reflexo da aplicação de princípios sólidos de design, como o Princípio da Responsabilidade Única (SRP).

Recomendação: É altamente recomendado que o código seja revisado e refatorado ativamente para quebrar os arquivos muito extensos, garantindo que cada um tenha um foco claro e responsabilidade única.

Finding FND-008: Falta de uma ferramenta configurável de logging (Severidade: Informativa)

Descrição da Vulnerabilidade

O `console.log` é um padrão do JavaScript, contudo, a adoção de uma ferramenta de log mais configurável poderia aprimorar a segurança do registro de eventos do sistema.

Recomendação de Mitigação: Substituição de `console.log` por uma Biblioteca de Logging Estruturada

Sugere-se a substituição imediata de todas as instâncias de `console.log` por uma biblioteca de logging estruturada e madura.

Justificativa - Benefícios Imediatos

1. **Logging Mais Limpo e Contextual:** Permite a inclusão automática de informações contextuais (timestamp, arquivo/linha de origem, nome do módulo) e o registro de objetos de dados de forma estruturada (JSON).
2. **Filtragem de Dados Sensíveis (Sanitização):** Recurso crucial para prevenir o vazamento acidental de dados sensíveis (P.I.I., tokens de acesso, senhas parciais) através da configuração de *sanitizadores* ou *filtros*.

Justificativa - Flexibilidade Operacional

1. **Configuração de Nível de Log (Nível de Detalhamento):** Bibliotecas profissionais oferecem a facilidade de configurar o nível de detalhe do log (*loglevel*) por ambiente ou módulo (DEBUG, INFO, WARN, ERROR, FATAL).

A adoção desta prática eleva o padrão de segurança da aplicação ao mitigar riscos de vazamento de dados via logs, além de melhorar a manutenibilidade e a capacidade de diagnóstico (*observabilidade*).

Finding FND-009: Validação de Host no sshCollector.sh (Severidade: Informativa)

Descrição da Vulnerabilidade

Não se trata de um problema de segurança, mas esta alteração poderia adicionar mais uma validação de segurança no sistema.

Recomendação de Mitigação

É recomendável implementar a validação de host no *script* `sshCollector.sh`. Embora esta validação possa ser negligenciada no primeiro acesso, a alteração da chave em acessos subsequentes pode indicar que o host foi comprometido ou reinstalado, permitindo que a aplicação gere um alerta de segurança.

6. Conclusão e Próximos Passos

A análise de segurança de código-fonte da aplicação SamurEye é um passo fundamental para garantir a robustez e a segurança do produto. A remediação das vulnerabilidades listadas, em particular as de severidade Crítica e Alta, deve ser a prioridade imediata da equipe de desenvolvimento da GruppenIT.

Caso surjam dúvidas sobre quaisquer dos pontos, ou na eventualidade de serem solicitados exemplos de código para a correção dos itens mencionados, podemos agendar uma reunião ou posso detalhar qualquer ponto por meio de comunicação eletrônica (email).